Decision Record

63 decisions

Every accepted, superseded and deferred decision, in order

This is the complete architecture decision record for the homelab provisioning system, generated from the Markdown sources so the printed copy cannot drift from the repository. Each entry carries its status and its Decision section verbatim. Context and consequences remain in `homelab/decisions/`.

Accepted	45
Deferred for post-proof revalidation	14
Superseded	4

ADR 0001 Local repository location
Superseded by ADR 0046 · 2026-07-24

Use `/home/ksh/git/codex/homelab-infrastructure` as the normal local working copy and initialize it as a Git repository with `main` as its initial branch.

ADR 0002 Initial computer profiles
Accepted · 2026-07-24

Begin with two computer profiles:

- **Controller**, identified as `controller`
- **Workstation**, identified as `workstation`

The Controller is the homelab server, orchestrator, and general infrastructure utility. Each Workstation deployment pivots on the hostname of its Controller.

ADR 0003 Design-phase commit granularity
Accepted · 2026-07-24

During the initial architecture and design phase, accumulate and integrate related decisions into larger coherent sections before committing them. After the design foundation exists, use smaller, focused commits for implementation work and later revisions.

ADR 0004 Interactive install authorization
Accepted · 2026-07-24

Do not require a pre-staged deployment record. The network-boot environment will collect and validate several operator inputs at the target computer, display a complete preflight summary, and perform no destructive operation until the operator gives a final explicit confirmation. Physical-console and boot-path access, together with this interactive confirmation workflow, are the authorization boundary.

ADR 0005 Use home.arpa for internal DNS
Accepted · 2026-07-24

Use `home.arpa` as the internal DNS suffix. Controller references use a fully qualified hostname beneath it, such as `polycarp.home.arpa`.

ADR 0006 Keep baseline and bootstrap DNS on UniFi
Superseded by ADR 0008 · 2026-07-24

Keep UniFi as the normal client-facing DNS endpoint and the owner of the Controller's bootstrap record. For a Controller whose instance hostname is `polycarp`, that record is `polycarp.home.arpa`. Defer Controller-hosted DNS. A later design may place an authoritative child zone on the Controller and configure a UniFi Forward Domain rule for that child zone, but it must preserve the UniFi-hosted Controller bootstrap record.

ADR 0007 Design generic profiles and build a Controller first
Accepted · 2026-07-24

Design reusable Controller and Workstation profiles without using an inventory of the current Polycarp installation as an architecture input. The first end-to-end acceptance test will network boot a target and use the generated provisioning system to build a Controller from bare metal. The acceptance sequence will then validate Controller-owned DHCP and DNS on an isolated network whose only permanent network infrastructure is a switch.

ADR 0008 Support Controller-owned DHCP and DNS
Accepted · 2026-07-24

The Controller must optionally function as the primary DHCP and DNS server for its managed network. When external infrastructure owns DHCP or DNS, the corresponding Controller services must remain disabled. Two DHCP authorities must never be active on the same layer-2 network during installation, testing, or transition. The first acceptance sequence must validate the installed Controller's DHCP and DNS behavior on an isolated network containing no permanent router or DNS/DHCP appliance. Provisioning occurs first on the existing full network, and ADR 0010 defines the powered-off transition to the isolated network.

ADR 0009 Activate Controller network services automatically
Accepted · 2026-07-24

When the Controller is provisioned for Controller-owned DHCP and DNS, activate those services automatically on its first boot after provisioning. Do not require a separate manual activation command. The activation path must fail closed: if its required transition conditions cannot be verified, Controller DHCP must remain stopped.

ADR 0010 Power off before the network-service first boot
Accepted · 2026-07-24

When Controller-owned DHCP and DNS are selected, a successful installation must end with a clean poweroff rather than rebooting into the installed operating system. The operator relocates the powered-off Controller to the isolated switch and powers it on. That power-on is the first boot of the installed system, and ADR 0009 automatically activates the configured network services after validation.

ADR 0011 Exclude routing and NAT from the initial Controller
Accepted · 2026-07-24

Exclude routing, NAT, and perimeter-firewall duties from the initial Controller profile. On the isolated acceptance network, the Controller provides local DHCP and DNS without advertising or acting as a default gateway. The initial isolated test therefore has no Internet access. ADR 0045 subsequently excludes a gateway from the network input schema and derives the absence of the DHCP default-router option. Reserve a clearly marked future-extension section in the Controller design for possible routing and NAT support.

ADR 0012 Bundle Controller DHCP and DNS
Accepted · 2026-07-24

Expose one combined Controller network-services option in the initial profile:

- When enabled, Controller DHCP and DNS are configured and activated together.
- When disabled, both remain stopped and external infrastructure provides those functions.

Reserve independent DHCP-only and DNS-only operation as a future extension. ADR 0044 subsequently selects one host-native dnsmasq instance as the initial implementation of this bundle.

ADR 0013 Start Controller network services with IPv4
Accepted · 2026-07-24

Limit the initial bundled Controller network-services option to managed IPv4:

- provide DHCPv4;
- serve DNS on the Controller's IPv4 service address;
- do not provide DHCPv6, router advertisements, or managed IPv6 addressing.

Keep the operating system's IPv6 stack and ordinary link-local behavior enabled. Reserve managed dual-stack operation as a future extension.

ADR 0014 Use a host-first hybrid Controller
Accepted · 2026-07-24

Use a host-first hybrid architecture:

- The Controller host operating system directly owns boot, storage, physical networking, first-boot safety, bundled DHCP/DNS, PXE control, and local recovery.
- Use containers when they materially simplify packaging or independent service lifecycle.
- Use VMs only when a service requires a separate kernel, different operating system, or stronger trust boundary.
- Do not require virtualization for every service.

ADR 0015 Use Arch Linux for the Controller host
Accepted · 2026-07-24

Use Arch Linux as the host operating system for the initial Controller profile. This decision selected the operating-system family only. Subsequent ADRs select the kernels and Btrfs-inside-LUKS2 root filesystem. The installation package state and complete rollback mechanism remain open. FreeBSD is not part of the initial Controller baseline. It remains available for a future profile or for a workload that specifically benefits from it.

ADR 0016 Use guarded direct Arch updates
Accepted · 2026-07-24

Use pacman directly for guarded Controller updates:

- do not apply unattended operating-system package upgrades;
- review Arch news and the complete pending update set before maintenance;
- create a recoverable system checkpoint before changing packages;
- perform a supported full-system pacman upgrade, not a partial upgrade;
- reboot when the transaction affects the kernel, initramfs, system manager, networking stack, or foundational services;
- automatically validate Controller health, including DHCP, DNS, and PXE, after the update; and
- restore the checkpoint or rebuild from recorded inputs if validation fails.

Do not create a private Arch repository, a package-promotion service, or a wrapper package manager for the initial Controller. This ADR defines the workflow boundary, not its implementation. Subsequent ADRs select the kernels and Btrfs-inside-LUKS2 root filesystem, and ADR 0028 selects native Btrfs checkpoint operations. ADR 0032 requires the off-host Secure Boot signing path to pass preflight before a hardened full-system upgrade mutates packages. ADR 0043 defers that signing requirement during the functional proof: the proof workflow must instead regenerate, checksum, install, and validate the matching development UKIs as part of the guarded transaction. Exact health checks, maintenance cadence, retention, and package-state recording format remain undecided.

ADR 0017 Use linux-lts as the default Controller kernel
Accepted · 2026-07-24

Use Arch's official `linux-lts` package as the default kernel for the initial Controller profile. This decision did not determine whether the profile also installs a secondary kernel; ADR 0018 subsequently selected the standard `linux` package for that role. This decision does not alter the guarded full-system update policy in ADR 0016.

ADR 0018 Install the standard Linux kernel as a secondary kernel
Accepted · 2026-07-24

Install Arch's official standard `linux` package as the secondary Controller kernel. Keep `linux-lts` as the default boot selection. Both kernels are managed by pacman and updated in the same guarded full-system transaction defined by ADR 0016. ADR 0022 subsequently selects systemd-boot. The eventual configuration must expose an operator-selectable standard-kernel entry without making it the automatic default; exact menu behavior remains open.

ADR 0019 Require UEFI boot for the Controller
Accepted · 2026-07-24

Require the initial Controller profile to provision and boot in UEFI mode. Legacy BIOS and Compatibility Support Module boot are unsupported. The installer preflight must verify that it is running in UEFI mode before offering destructive authorization. The eventual installed-system acceptance test must verify UEFI boot with `linux-lts` as the default and standard `linux` available as the secondary kernel. This decision applies to the Controller profile. It does not independently decide the firmware requirements for every future Workstation target. ADR 0020 subsequently requires Secure Boot and encryption at rest. ADR 0030 retains factory PK and KEK ownership while adding the lab signer to `db`. Disk partitioning and the precise UEFI network-boot chain remain separate decisions. ADR 0022 subsequently selects systemd-boot for Arch, and ADR 0023 selects signed Unified Kernel Images.

ADR 0020 Require encryption at rest and authenticated boot
Accepted · 2026-07-24

For systems installed and managed by the Controller and Workstation profiles:

- encrypt every persistent data-bearing operating-system and service volume;
- encrypt disk-backed swap and persistent crash or hibernation state;
- encrypt backups of managed-system data, secrets, and recovery state;
- use LUKS2 for Linux data-bearing volumes;
- use BitLocker for Windows data-bearing volumes;
- require Secure Boot for provisioning, installed-system boot, and recovery paths;
- limit unencrypted storage to mandatory firmware-readable boot and recovery partitions;
- keep those unencrypted partitions minimal and free of credentials, decryption keys, tokens, private data, and other reusable secrets; and
- authenticate executable boot artifacts through the accepted Secure Boot chain.

This is an at-rest encryption boundary. It does not yet define transport encryption. The policy applies to new or rebuilt systems managed by these profiles. It does not assert that existing lab devices have already been migrated or authorize inventorying them. ADR 0043 creates one narrow sequencing exception. Its labeled, disposable `development-proof` installation must still use encryption at rest, but may defer the custom authenticated-boot chain and run with Secure Boot disabled when an already trusted upstream chain is unavailable. Such an installation does not satisfy this ADR for final profile acceptance and must be reprovisioned after the signing design is revisited.

ADR 0021 Install a permanent Windows maintenance environment
Superseded by ADR 0047 · 2026-07-24

Include a permanent, licensed, bare-metal Windows maintenance installation in the initial Controller profile.

- Use Windows only for physical firmware and hardware maintenance.
- Keep Arch Linux as the default boot and the sole normal Controller runtime.
- Do not host Controller services or authoritative configuration in Windows.
- Protect the Windows operating-system volume with BitLocker.
- Keep Windows Boot Manager independently registered as a native UEFI boot option.
- Store the BitLocker recovery material off-host under the recovery policy required by ADR 0020.
- Treat WinPE and vendor-supplied boot media as supplemental recovery or one-off tools, not as the supported primary Windows maintenance path.

The Windows edition, license source, storage allocation, account model, update cadence, direct-boot user experience, and exact relationship between Windows Boot Manager and the Linux boot manager remained undecided here. ADR 0022 subsequently selects systemd-boot for Arch while preserving an independent native Windows UEFI entry; the direct-boot user experience remains open. ADR 0043 permits the Windows installation and direct UEFI entry to be exercised during the functional proof. That test is not final Windows acceptance; BitLocker, Secure Boot, TPM, and recovery behavior must be completed and retested after boot hardening.

ADR 0022 Use systemd-boot for Arch
Accepted · 2026-07-24

Use systemd-boot as the installed Arch boot manager for the initial Controller profile.

- Make the systemd-boot UEFI entry the normal firmware default.
- Make `linux-lts` the automatic Arch kernel selection.
- Expose standard `linux` as an operator-selectable secondary Arch entry.
- Keep Windows Boot Manager independently registered as a native UEFI entry.
- Do not make Windows depend on systemd-boot remaining functional.
- Sign and trust systemd-boot as part of the Secure Boot chain required by ADR 0020.

The operator experience for requesting a Windows boot remains undecided. systemd-boot may discover Windows, but the final design must retain a direct firmware path that does not require chainloading through the Linux boot manager. ADR 0023 subsequently selects signed Unified Kernel Images as the kernel artifact format. ADR 0030 retains factory PK and KEK ownership while adding the lab signer to `db`, and ADR 0031 makes that signer lab-wide. Private-key custody is subsequently resolved by ADR 0032. Signing automation, boot counting, checkpoint integration, and boot-menu timeout remain undecided. ADR 0024 subsequently adds a self-contained recovery UKI, and ADR 0025 places all boot artifacts on one shared ESP without XBOOTLDR. Under ADR 0043, Milestone A may exercise the same systemd-boot and UKI topology without the custom trust and signing workflow. Signing remains mandatory for final acceptance after the deferred design is revalidated.

ADR 0023 Use signed Unified Kernel Images
Accepted · 2026-07-24

Build and install a signed UKI for each accepted Arch kernel:

- one UKI for `linux-lts`, selected by default; and
- one UKI for standard `linux`, exposed as the secondary Arch entry.

Each UKI contains its matching kernel, initramfs, and kernel command line. systemd-boot discovers and launches the UKIs as Boot Loader Specification Type #2 entries. Do not make an unsigned or partially authenticated Linux boot entry part of the normal installed-system path. The Secure Boot trust chain must verify each UKI before execution. UKIs reside on firmware-readable, unencrypted boot storage and are therefore public artifacts. They must not contain raw volume keys, credentials, reusable tokens, private data, or other secrets. This decision does not select the UKI generator, initramfs generator, signing tool, or signing automation. ADR 0024 subsequently selects a third self-contained recovery UKI, ADR 0025 selects one shared ESP for all boot artifacts, ADR 0029 defines TPM2 unlock authorization for the two normal UKIs, ADR 0030 defines additive enrollment of the lab signer without replacing factory PK or KEK ownership, and ADR 0032 defines removable off-host custody of the signing private key. ADR 0043 keeps the UKI topology but allows checksum-verified, unsigned development UKIs only in the explicitly non-production functional proof. That exception is removed before final acceptance; it is not an installed profile option.

ADR 0024 Include a self-contained recovery UKI
Accepted · 2026-07-24

Install a third, signed, self-contained recovery UKI on the Controller. The recovery UKI:

- boots independently of the installed Arch root filesystem;
- is operator-selectable and never the automatic normal boot;
- contains the storage, encryption, filesystem, and boot-artifact tools needed to diagnose and recover the installed system;
- requires separately held LUKS2 recovery material under ADR 0029 and contains no raw unlock key, credential, token, or other reusable secret;
- starts in a non-destructive state and requires explicit operator action before mounting writable storage or changing disk state; and
- is authenticated by the same accepted Secure Boot trust policy as the normal Arch UKIs.

The exact rescue userspace, image generator, included hardware support, network behavior, update cadence, validation procedure, and whether firmware also registers the recovery UKI directly remain undecided. ADR 0043 permits a checksum-verified, unsigned recovery-UKI candidate during the functional proof so its userspace and recovery behavior can be exercised. Its authenticated execution, rotation, and interaction with TPM policy remain final-hardening acceptance tests.

ADR 0025 Use one shared EFI System Partition
Accepted · 2026-07-24

Use one FAT32 EFI System Partition shared by Windows and Arch on the initial Controller.

- Store Windows Boot Manager in its standard vendor directory.
- Store systemd-boot in its standard vendor directory.
- Store the two normal Arch UKIs and the recovery UKI in the Boot Loader Specification location on the ESP.
- Do not create an XBOOTLDR partition.
- Keep the ESP unencrypted, minimal, secret-free, and covered by Secure Boot integrity verification under ADR 0020.
- Back up and validate the complete ESP as one unit.

ADR 0043 preserves this physical layout during the functional proof but defers Secure Boot integrity acceptance. Proof-stage ESP backups and artifacts use checksums and explicit `development-proof` labeling until hardening. The exact ESP size must be selected only after measuring the generated UKIs and allowing documented update and recovery headroom. Fallback-path ownership, redundant boot-media copies, backup format, and restore procedure remain undecided.

ADR 0026 Use Btrfs inside LUKS2 for the Arch root
Accepted · 2026-07-24

Use Btrfs for the initial Controller's Arch root filesystem, contained inside a LUKS2-encrypted volume. This decision selects the root filesystem and encryption layering only. ADR 0027 subsequently defines the operating-system checkpoint boundary. The following remain open:

- compression and mount options;
- swap implementation;
- service-data placement and application-consistent backup behavior;
- multi-device layout or redundancy;
- storage-capacity allocations.

ADR 0028 subsequently selects native Btrfs operations for checkpoints. ADR 0029 subsequently defines automatic TPM2 root unlock plus an off-host recovery key. Additional persistent data-bearing volumes must still satisfy ADR 0020 but do not automatically inherit this exact filesystem choice.

ADR 0027 Limit root checkpoints to operating-system state
Accepted · 2026-07-24

Make the root Btrfs checkpoint an operating-system checkpoint. The checkpointed root includes:

- installed operating-system files and libraries;
- `/etc` and other declarative host configuration;
- pacman's database and other package-manager state; and
- system state that must remain version-aligned with the installed packages.

Use separate Btrfs subvolumes, excluded from a root checkpoint, for:

- ordinary user home directories and the root operator's home;
- logs;
- caches and disposable temporary state;
- the snapshot store itself; and
- mutable application and Controller service data.

Do not exclude all of `/var/lib` as one unit. Keep pacman and other version-coupled operating-system state with the root, and assign mutable service paths to separate subvolumes when each service is defined. The exact subvolume names, mount options, retention rules, and service paths remain undecided. ADR 0028 subsequently selects native Btrfs operations as the checkpoint primitive.

ADR 0028 Manage checkpoints with native Btrfs operations
Accepted · 2026-07-24

Use native Btrfs subvolume operations as the checkpoint primitive in the repository-controlled guarded-update workflow.

- Create a read-only snapshot of the operating-system root before a guarded update.
- Associate that root snapshot with a backup and manifest of the matching shared-ESP artifacts.
- Record enough update and package-state metadata to identify the checkpoint and the transaction it protects.
- Use the self-contained recovery UKI to restore a root checkpoint and its matching boot artifacts when the installed root is not safely recoverable.
- Do not install Snapper, `snap-pac`, or an automatic timeline-snapshot service for the initial Controller.

This decision selects the checkpoint primitive rather than the complete workflow. Snapshot naming, metadata format, retention limits, deletion policy, ESP backup format, exact rollback commands, and post-validation checkpoint lifecycle remain undecided.

ADR 0029 Automatically unlock the Controller root with TPM2
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

ADR 0043 defers this decision during the functional proof. Do not enroll a TPM unlock token or policy in Milestone A; unlock LUKS2 manually. Revisit this ADR with ADRs 0030 through 0042 after the functional environment works. Require TPM 2.0 for the initial Controller and automatically unlock its Arch LUKS2 root only from either normal signed UKI:

- the default `linux-lts` UKI; or
- the secondary standard `linux` UKI.

Bind the TPM enrollment to a signed policy over the normal UKIs' measured boot state, using the systemd UKI measurement in PCR 11. Do not rely on an unconstrained TPM enrollment or only on the current raw PCR values. The exact additional PCR bindings remain undecided. ADR 0033 subsequently selects a dedicated TPM-policy signing identity distinct from the Secure Boot signer and applies ADR 0032's removable off-host custody pattern to its private key. ADR 0034 subsequently shares that policy identity across Controllers without sharing their TPM-sealed LUKS2 secrets. Enroll a separate, high-entropy LUKS2 recovery key. Store it off-host and never in Git, the ESP, a UKI, ordinary logs, or the Controller's only recovery copy. If TPM policy validation fails, early boot must fall back to recovery-key entry without weakening or replacing the policy automatically. Authenticate the recovery UKI with Secure Boot, but do not give it the signed PCR authorization needed for automatic LUKS2 unlock. Recovery boot always requires separately held recovery material. Do not require a TPM PIN for normal Controller boot. ADR 0030 subsequently retains OEM and Microsoft firmware trust while adding a

lab signer to Secure Boot `db`. That broader permission to execute an EFI image does not broaden this TPM policy: only the two approved normal lab UKI measured states receive automatic root unlock. ADR 0031's lab-wide Secure Boot signing identity is not thereby selected as the TPM-policy signing identity; ADR 0033 subsequently requires a separate key pair.

ADR 0030 Add lab Secure Boot trust without replacing platform ownership
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Use an additive Secure Boot trust model for the initial Controller profile.

- Retain the platform's factory or OEM ownership represented by its PK and KEKs. Do not replace them with lab-owned PK or KEK material.
- Preserve the compatible OEM and Microsoft trust required for Windows, firmware servicing, third-party UEFI applications, and option ROMs.
- Preserve the platform's supported `dbx` revocation-update path.
- Add a lab-controlled public signing certificate to the firmware's authorized signature database (`db`) through a firmware-supported, physical-presence enrollment procedure that preserves the existing trust entries.
- Sign systemd-boot and all three Controller UKIs with the corresponding lab signing identity.
- Continue to authorize automatic LUKS2 unlock only for the two normal lab-approved UKIs under ADR 0029. Firmware permission to execute another OEM- or Microsoft-trusted EFI image does not grant that image the TPM unlock capability.

Before destructive installation, preflight must establish that the target can retain the required factory trust and enroll the lab certificate without replacing it. If that cannot be demonstrated, the initial Controller profile does not support the target and installation must stop before wipe authorization. Record the enrolled certificate fingerprints and the retained trust state in the machine's non-secret provisioning record. Do not put private signing material in firmware, the ESP, a UKI, or Git. ADR 0031 subsequently selects one lab-wide Secure Boot signing identity. The private-key custody model is subsequently resolved by ADR 0032. Signing automation, rotation and revocation procedure, exact enrollment tooling, and current certificate manifest remain separate decisions. The manifest must account for vendor and Microsoft certificate transitions rather than permanently assuming the factory's original certificate set is sufficient.

ADR 0031 Use one lab-wide Secure Boot signing identity
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Use one dedicated, lab-wide Secure Boot code-signing identity for managed Controller and Workstation systems.

- Enroll the same public certificate in firmware `db` on every managed machine that must execute lab-signed EFI artifacts.
- Use its private counterpart to sign the lab-controlled installed-system EFI artifacts defined by each profile, including the Controller's systemd-boot executable and normal and recovery UKIs.
- Record and verify the shared certificate fingerprint in the profile's non-secret artifact manifest and each machine's provisioning record.
- Do not create or manage a separate Secure Boot signing identity for every machine.

Sharing the identity does not mean copying its private key to every machine. ADR 0020 keeps private signing material off-host, and ADR 0032 subsequently selects encrypted removable primary and backup media used only in a designated signing environment. Signing automation, detailed rotation, and operator authorization remain separate decisions. This identity is scoped to Secure Boot code signing. This decision does not authorize its use for TLS, SSH, package repositories, backup encryption, disk recovery, or other unrelated purposes. ADR 0033 subsequently selects a different identity for signing TPM PCR policies. The network-provisioning trust chain remains open.

ADR 0032 Keep the Secure Boot private key on removable off-host media
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Keep the lab-wide Secure Boot private key on a primary encrypted removable medium that is normally detached and powered off. Use that medium only in a designated trusted signing environment during planned provisioning and update work. The existing bootstrap machine may provide the initial signing environment, but the key must not be mounted in its ordinary PXE-serving context. The exact isolated or ephemeral signing environment remains an implementation decision. If that bootstrap machine is later provisioned as a managed Controller or Workstation, signing responsibility must move elsewhere first. Do not copy or persist the private key on:

- a Controller or Workstation;
- an EFI System Partition or UKI;
- a PXE root, installer, or target machine;
- the ordinary filesystem of the bootstrap host;
- an always-online signing service; or
- Git, logs, build output, or provisioning records.

Build or prepare artifacts outside the key medium, validate the artifact and its non-secret manifest in the signing environment, sign it there, verify the result against the expected public-certificate fingerprint, and return only the signed artifact and non-secret metadata. Unsigned candidates must remain outside the active ESP. Package or boot-manager hooks must not install an unsigned or locally self-signed replacement into the active boot path. Create one separately encrypted backup copy on a second removable medium. Keep it offline and physically separate from the primary medium and from the credential that unlocks either copy. Do not connect the primary and backup media during the same routine signing session. Periodically verify that the backup can be decrypted and can produce a signature accepted by the recorded public certificate; the exact test cadence remains open. Every guarded Controller full-system upgrade must preflight the signing path before package mutation, because the transaction may require new systemd-boot, kernel, initramfs, or UKI signatures. Absence or failure of the signer stops the update before intentional changes begin. A boot-affecting update is not successful until all returned artifacts pass signature and manifest validation and are installed together. Normal boot and ordinary runtime do not require access to the private key. ADR 0036 subsequently selects manually unlocked LUKS2 volumes for both removable media. The signing environment, transfer format, tooling, temporary-data handling, and detailed rotation and revocation procedure remain separate decisions. ADR 0040 subsequently selects the operator's existing off-Controller password manager for unlock-credential custody. ADR 0033 subsequently applies this custody pattern to a distinct TPM-policy signing private key. ADR 0035 subsequently places both distinct private keys on the same physical primary medium and both backup copies on the same physical backup medium.

ADR 0033 Separate TPM policy signing from Secure Boot signing
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Create a dedicated TPM signed-PCR policy key pair that is cryptographically distinct from the lab-wide Secure Boot code-signing key pair. Use the TPM policy private key only to sign approved PCR 11 policies for the two normal Controller UKIs:

- the default `linux-lts` UKI; and
- the secondary standard `linux` UKI.

Do not give the recovery UKI a policy signature that authorizes automatic root unlock. Continue to sign it only for Secure Boot execution under ADR 0024. Enroll or otherwise bind only the TPM policy public key to the Controller's LUKS2 TPM2 unlock policy. The public key and its fingerprint are non-secret and belong in the artifact manifest and machine provisioning record. Never enroll, embed, or copy the TPM policy private key to a managed machine. Apply ADR 0032's custody pattern to the TPM policy private key: keep encrypted primary and backup copies on removable off-host media and use the key only in the designated signing environment during planned work. ADR 0035 subsequently co-locates both distinct private keys on the same primary medium and both backup copies on the same backup medium. For each normal UKI, the returned artifact must contain a valid TPM policy signature and a valid Secure Boot signature. The final Secure Boot signature must authenticate the UKI containing its policy metadata. Verify both authorities before installing the artifact in the active ESP. ADR 0034 subsequently makes the TPM policy identity shared across Controllers. ADR 0035 subsequently resolves its physical-media placement. Its algorithm, lifetime, exact systemd enrollment and signing commands, rotation procedure, and additional PCR bindings remain separate decisions.

ADR 0034 Share one TPM policy signing identity across Controllers
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Use one dedicated TPM policy signing identity across all machines provisioned with the Controller profile.

- Bind the same TPM policy public key to each Controller's LUKS2 signed-PCR policy.
- Use the corresponding shared private key to sign approved PCR 11 policies for each Controller's two normal UKIs, including instance-specific UKIs.
- Record and verify the common policy-public-key fingerprint in the Controller profile artifact manifest and each instance's non-secret provisioning record.
- Do not generate or maintain a separate TPM policy private key for every Controller.

This does not create a shared LUKS2 recovery key, volume key, TPM-sealed secret, or TPM enrollment. Those remain unique to each Controller. Moving an encrypted disk to another Controller does not move its automatic-unlock capability. The identity remains cryptographically separate from the lab-wide Secure Boot signer under ADR 0033 and follows ADR 0032's removable off-host custody model. ADR 0035 subsequently places both private keys on the same primary removable medium and both backup copies on the same backup medium. This decision applies only to the accepted Controller automatic-unlock design. It does not enroll Workstations or future profiles into the policy without a separate decision.

ADR 0035 Co-locate both signing keys on each removable medium
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Store both distinct private keys on the same encrypted primary removable medium:

- the lab-wide Secure Boot code-signing private key; and
- the Controller-wide TPM-policy signing private key.

Store the backup copy of both keys together on the same separately encrypted backup medium defined by ADR 0032. Keep the identities logically and cryptographically separate. Give each key a role-specific name, expected public-key fingerprint, and explicit signing configuration. Signing tooling must select the intended key by role rather than expose one ambiguous generic signer. Rotating either identity must update and verify both the primary and backup media before the rotation is complete. The other identity does not rotate merely because the two share storage. ADR 0036 subsequently selects LUKS2 for both removable media. The filesystem, subsequently selected by ADR 0037, is ext4. The directory layout, file encryption and permissions, and detailed backup-synchronization procedure remain separate decisions. ADR 0039 subsequently selects one shared passphrase for both media, and ADR 0040 places its authoritative custody in the operator's existing off-Controller password manager.

ADR 0036 Encrypt signing media with LUKS2
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Use one LUKS2-encrypted data volume on each signing medium:

- one independently encrypted LUKS2 volume on the primary medium; and
- one independently encrypted LUKS2 volume on the backup medium.

Place all private signing material on the filesystem inside that encrypted volume. Do not create an unencrypted data filesystem or private-key copy elsewhere on either medium. The partition table and LUKS2 header are mandatory unencrypted metadata and must contain no secrets beyond the encrypted keyslot material intrinsic to LUKS2. Require an operator-supplied unlock credential in the designated signing environment. Do not configure TPM auto-unlock, host-stored keyfiles, or another automatic unlock path for either removable volume. After signing, unmount the filesystem, close the LUKS mapping, and physically detach the medium. The signing workflow must fail closed if either operation does not complete cleanly. Record the non-secret volume identifiers and the exact cryptographic and key derivation parameters used to create each volume. Do not expose an unlock credential in that record. ADR 0037 subsequently selects ext4 as the inner filesystem. The partition layout, LUKS2 cipher and PBKDF parameters, LUKS header backup procedure, file-level key encryption, and rekey procedure remain separate decisions. ADR 0038 subsequently selects the routine mount policy, and ADR 0039 selects one shared manually entered passphrase. ADR 0040 subsequently places its authoritative custody in the operator's existing off-Controller password manager.

ADR 0037 Use ext4 inside the signing-media LUKS2 volumes
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Create one ext4 filesystem inside each signing-media LUKS2 volume:

- one independently created ext4 filesystem on the primary medium; and
- one independently created ext4 filesystem on the backup medium.

Store private keys, public certificates, fingerprints, and non-secret role-specific metadata as ordinary files with Unix ownership and permissions. Do not use FAT, exFAT, Btrfs, ZFS, or another inner filesystem for the initial signing-media design. Give the two filesystems distinct, non-secret UUIDs and labels so tooling can verify that the operator attached the intended primary or backup medium before opening or synchronizing it. Do not use a device path such as `/dev/sdX` as identity. The exact labels, filesystem creation options, directory layout, ownership, permissions, mount options, free-space threshold, check cadence, and backup-synchronization procedure remain separate decisions. ADR 0038 subsequently selects read-only routine mounts with `ro,noload,nodev,nosuid,noexec` and reserves writable mounts for explicit signing-media maintenance.

ADR 0038 Mount signing media read-only during routine signing
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

During routine signing:

- open the LUKS2 mapping read-only;
- verify the expected LUKS2 and ext4 identities;
- require the ext4 filesystem to be clean before mounting it;
- mount it with `ro,noload,nodev,nosuid,noexec`;
- read private keys only through their role-specific configured paths; and
- write signing requests, signatures, completed artifacts, logs, and temporary files somewhere other than the signing medium.

If the filesystem is dirty, damaged, unexpectedly writable, or has the wrong identity, stop signing. Do not replay its journal or repair it implicitly in the routine path. Permit writes to the decrypted volume, including a read-write mount, only during an explicit signing-media maintenance operation, such as:

- initial key creation;
- an authorized key or certificate rotation;
- a verified metadata change;
- primary-to-backup synchronization; or
- filesystem repair.

A writable operation must identify whether it targets the primary or backup, validate the pre-change key fingerprints and manifest, produce and validate a post-change manifest, flush writes, cleanly unmount ext4, close LUKS2, and detach the medium before it is complete. Keep the backup detached except during planned synchronization, read-only recovery testing, or repair. Routine trial-signature tests use the same read-only policy as the primary. The exact mount point, filesystem-cleanliness command, writable-maintenance authorization, staging location, manifest format, synchronization procedure, and temporary-data cleanup remain separate decisions.

ADR 0039 Unlock both signing media with one shared passphrase
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Generate one high-entropy random passphrase and enroll it in a LUKS2 keyslot on both signing-media volumes.

- Generate at least 128 bits of entropy with a cryptographically secure random source and encode it in a form suitable for reliable manual entry or transfer.
- Use the same passphrase for the primary and backup, while retaining the independently generated LUKS2 volume master key on each medium.
- Enter the passphrase interactively in the designated signing environment.
- Do not store it on either signing medium, any Controller or Workstation, the bootstrap or signing host, an installer, PXE storage, Git, a command line, environment variable, shell history, log, or generated artifact.
- Keep its authoritative custody off-host and physically separate from both signing media.

Changing the shared passphrase is incomplete until the new credential has been enrolled and tested on both media and the old credential has been removed from both. Test each medium separately. ADR 0040 subsequently selects the operator's existing off-Controller password manager as the authoritative store. The passphrase generator and encoding, human access policy, input method, keyslot layout, LUKS2 PBKDF parameters, and rotation cadence remain separate decisions. ADR 0041 subsequently requires one independent emergency copy, and ADR 0042 selects its sealed paper form.

ADR 0040 Store the signing-media passphrase in an off-Controller password manager
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Use the operator's existing off-Controller password manager as the authoritative store for the shared signing-media passphrase. The password manager used for this purpose must:

- remain accessible when the Controller is powered off, damaged, isolated, or being rebuilt;
- not depend exclusively on DNS, identity, networking, or storage supplied by the Controller;
- protect its local or remote vault independently of the signing media; and
- have an operator-tested recovery path independent of the Controller.

Retrieve the passphrase manually for a signing session and enter it interactively into the designated signing environment. Do not add password manager API credentials, automated vault retrieval, or a machine account to the initial signing workflow. Do not cache or persist the retrieved passphrase on the signing host. The repository and non-secret machine records may contain only a stable, non-secret reference describing which password-manager entry the operator must retrieve. They must not contain the passphrase, a vault export, session token, recovery code, or password-manager credential. The specific password-manager product and account remain operator-owned instance details rather than Controller profile dependencies. This design does not require external registration; a qualifying local or externally hosted manager is acceptable. ADR 0041 subsequently requires one independent emergency copy, and ADR 0042 selects a sealed paper recovery sheet in locked, fire-resistant storage. Password-manager entry naming, clipboard versus direct-entry handling, authorized human access, audit cadence, and coordinated passphrase-rotation procedure remain separate decisions.

ADR 0041 Keep one independent emergency copy of the signing-media passphrase
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Maintain exactly one authorized emergency copy of the shared signing-media passphrase outside the password manager. The emergency copy must:

- reproduce the exact passphrase without depending on memory;
- remain offline and require no Controller, home-network, DNS, identity, cloud, or password-manager service;
- be stored physically apart from the password manager's recovery material, the primary signing medium, and the backup signing medium;
- never appear in Git, ordinary documentation, an inventory database, a Controller or Workstation, PXE storage, an installer, or a signing host;
- be accessed only for password-manager recovery or a scheduled break-glass test; and
- be returned to protected storage or securely replaced immediately after use.

The password manager remains the normal authoritative source. The emergency copy is a break-glass recovery path, not a second routine retrieval location. A passphrase rotation is incomplete until the password manager, both LUKS2 volumes, and the emergency copy agree on the new credential and each has been tested through its appropriate path. Destroy the obsolete emergency copy only after the new one has been verified. ADR 0042 subsequently selects a human-readable paper recovery sheet in a tamper-evident sealed envelope kept in locked, fire-resistant storage. The exact storage location, access control, test cadence, and destruction method remain separate decisions.

ADR 0042 Use a sealed paper emergency passphrase copy
Deferred for post-proof revalidation by ADR 0043 · 2026-07-24

Print the exact signing-media passphrase as a human-readable recovery sheet. Place it in a tamper-evident sealed envelope and keep the envelope in locked, fire-resistant storage physically separate from:

- the primary signing medium;
- the backup signing medium; and
- the password manager's own recovery material.

The sheet may contain only:

- the exact passphrase in a transcription-resistant layout;
- a non-secret purpose and version label;
- enough non-secret instructions to identify the two signing-media LUKS2 volumes it unlocks; and
- a verification value that detects transcription error without containing another secret.

Do not print either signing private key, a password-manager credential, a vault recovery code, or unrelated recovery material on the sheet. Before sealing a new or rotated sheet, use it to unlock and close each medium separately, verify both key fingerprints through a trial signature, and confirm that no digital copy remains in the print path, spool, temporary directory, or device storage under operator control. Record only a non-secret envelope identifier, creation date, and seal-inspection status outside the envelope. Do not record the passphrase or an image of the sheet. Opening the envelope is a break-glass event. Record the event without the secret, replace the sheet and envelope after use, and rotate the passphrase if custody or observation during the event is uncertain. This plaintext physical sheet is an explicit exception to digital at-rest encryption. Its locked location, separation, tamper evidence, and access control form its protection boundary. The exact storage location, authorized people, paper and printing method, envelope type, verification-value format, inspection and test cadence, and secure destruction method remain separate decisions.

ADR 0043 Prove the functional environment before boot hardening
Accepted · 2026-07-24

Split delivery into two explicit milestones. ### Milestone A: functional proof Build and test the non-production provisioning path first:

- network boot into the interactive installer;
- collect, validate, summarize, and explicitly confirm all destructive inputs;
- record UEFI mode, Secure Boot capability and state, and TPM 2.0 availability without changing firmware trust or enrolling a TPM token;
- install the generic Controller profile with Arch, LUKS2, Btrfs, systemd-boot, the two normal kernels, and the accepted shared-ESP layout;
- exercise the permanent Windows maintenance installation and its independent UEFI entry far enough to validate the functional dual-boot layout;
- power off after installation, relocate to the isolated switch, and perform the first installed boot;
- after manual LUKS2 unlock, start Controller DHCP and DNS automatically;
- verify isolated-network leases, `home.arpa` resolution, PXE service, and the no-routing/no-NAT boundary;
- exercise the Workstation profile's Controller-FQDN pivot; and
- prove checkpoint, rollback, rebuild, and health-validation concepts without depending on an offline signer.

Continue to verify the provenance and published checksums of downloaded installer and package artifacts even though Milestone A does not prove the final authenticated-boot chain. Milestone A does not create, enroll, or use a custom lab Secure Boot identity, TPM policy identity, TPM unlock token, signing medium, or signing credential. It does not clear factory firmware keys, enter firmware Setup Mode, alter firmware `db`, or implement TPM automatic LUKS2 unlock. If the Arch or network-boot path cannot use an already trusted upstream chain, Milestone A may run with Secure Boot disabled only under an explicit `development-proof` mode. That mode must:

- display and record that authenticated boot is not being tested;
- require a separate explicit operator acknowledgement before destructive authorization;
- use a manually entered LUKS2 passphrase at every Arch boot;
- contain no production secrets or irreplaceable data; and
- never be reported as a production-ready or fully accepted profile.

`development-proof` is a temporary project state recorded in the installation manifest, not a permanent profile option that can be selected to avoid hardening. Windows installation and direct UEFI boot may be tested in Milestone A, but final BitLocker, Secure Boot, TPM, and recovery behavior belongs to Milestone B. ### Milestone B: security hardening and final acceptance After Milestone A passes, revisit ADRs 0029 through 0042 as a group. They are design input, not an instruction to enroll TPM state or manufacture keys, credentials, or signing media now. Simplify, change, or replace them based on the proven update and provisioning workflow. Milestone B must then:

- enforce ADR 0020's Secure Boot and encryption requirements;
- establish and test the selected network-boot, installed-boot, and recovery trust chains;
- implement and test the selected signing and recovery-key custody;
- implement TPM automatic unlock only after its recovery path works;
- complete Windows BitLocker and recovery acceptance; and
- prove that an unattended cold restart restores Controller DHCP, DNS, and PXE service; and
- reprovision from the proven installer for final acceptance rather than treating the development-proof installation as production.

No system passes final Controller or Workstation acceptance until Milestone B is complete.

ADR 0044 Use dnsmasq for initial Controller network services
Accepted · 2026-07-24

Use the Arch `dnsmasq` package as the initial host-native implementation of the Controller's bundled network-services option. When Controller network services are enabled:

- dnsmasq provides DHCPv4 and is the only address-assigning DHCP authority on the managed layer-2 network;
- dnsmasq serves the Controller's local `home.arpa` DNS data and eligible DHCP lease or reservation names;
- DHCP advertises the Controller as the DNS server and does not advertise a default router on the isolated network;
- dnsmasq supplies architecture-appropriate PXE information and a restricted, read-only TFTP service for the small first-stage boot program; and
- the service binds only to the explicitly selected managed interface and address.

When external infrastructure owns DHCP and DNS, keep the installed Controller's dnsmasq network-service instance stopped. Whether the temporary bootstrap host uses UniFi boot options or a separate dnsmasq ProxyDHCP configuration remains a later decision. Do not deploy Kea, BIND, Unbound, CoreDNS, or a separate TFTP daemon for the initial Controller network-services bundle. Serve iPXE scripts, kernels, initramfs images, WIMs, and other substantial artifacts over a separately selected HTTP(S) service rather than TFTP. Revisit the service split if the proven environment requires DHCP high availability, several routed networks and relays, database- or API-driven IP address management, independently operated DNS, or authoritative DNS features beyond dnsmasq's practical scope.

ADR 0045 Collect a minimal explicit Controller IPv4 network plan
Accepted · 2026-07-24

When the bundled Controller network-services option is enabled, collect these four machine-readable installation inputs:

- `managed_ipv4_cidr`: the canonical network address and prefix length for the managed subnet;
- `controller_ipv4_address`: the Controller's static service address;
- `dhcp_pool_start`: the first address in the inclusive dynamic pool; and
- `dhcp_pool_end`: the last address in the inclusive dynamic pool.

Derive rather than prompt for:

- the IPv4 netmask;
- the network and broadcast addresses;
- the client DNS-server address, which is `controller_ipv4_address`;
- the DNS suffix, which is `home.arpa`; and
- the absence of a default-router option.

Before destructive authorization, require all of the following:

- every input parses unambiguously as IPv4;
- `managed_ipv4_cidr` uses the actual network address rather than an address with host bits set;
- the subnet contains enough usable addresses for the Controller and pool;
- the Controller and both pool endpoints are usable unicast addresses inside the subnet;
- the pool start is not greater than the pool end;
- the Controller address is outside the inclusive DHCP pool; and
- the complete entered and derived network plan is displayed in the preflight summary.

Record the confirmed inputs and derived plan in the non-secret installation manifest. If Controller-owned DHCP and DNS are disabled, these managed-network inputs are not required; the external-network host-addressing policy remains a separate decision. Do not prompt for a netmask, broadcast address, DNS server, DNS suffix, or gateway separately.

ADR 0046 Keep the homelab record inside Telos and split public profile from private instance
Accepted · 2026-07-25

Move the homelab design record, implementation and documentation into the Telos repository, and split it in two:

- **Profile material is public.** Decision records, the generic Controller, Workstation and Services profiles, the PXE design, the installer, the Ansible roles, and both reconstruction manuals contain no instance data and publish to the Telos site.
- **Instance material is private.** Real hostnames, addresses, interface names, disk serials, DHCP reservations, share paths and per-machine inventory live in `homelab/instance/`, which is gitignored and never rendered into the site.

Documents that need an instance value read it from the private overlay at build time and fall back to a clearly marked placeholder when the overlay is absent. A published document must be complete and useful without the overlay.

ADR 0047 Remove the permanent Windows maintenance installation from the Controller
Accepted · 2026-07-25

Remove the permanent Windows installation from the Controller profile.

- Perform firmware maintenance with `fwupd` where the vendor publishes to LVFS.
- Otherwise use vendor-supplied bootable media, created on demand.
- The Controller disk carries one ESP, one LUKS2 container and nothing else.
- Preflight records firmware vendor, version and LVFS availability in the installation manifest so the maintenance path is known before it is needed.

This decision governs the Controller profile only. The Workstation profile may still install Windows; that is decided separately.

ADR 0048 Serve boot and installation artifacts over HTTP with nginx
Accepted · 2026-07-25

Use the Arch `nginx` package as the Controller's artifact service.

- Serve a single read-only document root, `/srv/http/boot`.
- Bind only to the managed interface and the Controller service address.
- Serve plain HTTP on the isolated proof network. iPXE's HTTPS support depends on build options and an embedded trust store, and the proof network has no certificate authority; artifact integrity comes from published checksums verified by the installer, not from transport.
- Publish a `manifest.json` beside the artifacts listing every file with its SHA-256. The installer verifies each artifact against it before use.
- Run as a distinct system user with no write access to the document root.

Do not use a general-purpose application server, and do not enable directory autoindex on the published root.

ADR 0049 Build the installer as an Archiso netboot image driving a Python installer
Accepted · 2026-07-25

Build the provisioning environment as a custom Archiso profile, booted over the network, running a Python installer from this repository.

- The image carries the tools the installer needs and nothing else: `parted`, `cryptsetup`, `btrfs-progs`, `dosfstools`, `arch-install-scripts`, `systemd-ukify`, `python`, `nginx`-less, no desktop.
- The installer is ordinary Python 3 using only the standard library, so it can be unit-tested off-target. `homelab/lib/` holds the logic; `homelab/bin/` holds the entry points.
- Only administrator **public** SSH keys are baked into the image, enabling a parallel SSH session for observation. No private key, token or credential is ever placed in an image.
- The image is built reproducibly from a pinned package list and a recorded mirror snapshot, and its SHA-256 is published in the artifact manifest.

Do not use WDS, MDT, or a vendor deployment suite.

ADR 0050 Select the managed interface explicitly and pin it by MAC address
Accepted · 2026-07-25

Select the managed interface as an explicit installation input.

- Preflight enumerates every physical Ethernet interface with its kernel name, permanent MAC address, current link state and observed speed.
- The operator selects one. There is no default and no automatic choice.
- The installer records the **permanent MAC address** as the identity and writes a systemd `.link` file pinning a stable name, `lan0`, to that MAC.
- dnsmasq, nginx and the static address configuration all reference `lan0`.
- First-boot activation verifies that `lan0` exists, carries the expected MAC and has carrier before starting network services, and fails closed otherwise.
- Wireless interfaces are not offered.

ADR 0051 Fix the Controller disk layout and ESP size
Accepted · 2026-07-25

Use this GPT layout on a single Controller disk:

#	Size	Type	Contents
1	2 GiB	EFI System (FAT32)	systemd-boot, three UKIs, update headroom
2	rest	Linux LUKS2	Btrfs, subvolumes per ADR 0027

- 2 GiB is deliberate headroom, not a measurement. It costs nothing on any plausible Controller disk and removes an entire class of failed-update outage. The ESP is the one partition that cannot be grown after the fact without moving everything after it.
- No XBOOTLDR, no Microsoft reserved partition, no Windows recovery partition.
- Swap is a file inside the encrypted Btrfs root, not a separate partition, so ADR 0020's encrypted-swap requirement is satisfied without a second LUKS container.
- Hibernation is disabled on the Controller. A machine whose purpose is to answer DHCP does not suspend.
- The installer refuses any disk smaller than 64 GiB and requires the operator to confirm the disk by model, serial and size, never by `/dev/sdX`.

ADR 0052 Build the first Controller from an existing workstation
Accepted · 2026-07-25

Use an existing Arch workstation as the temporary bootstrap host for the first Controller build.

- Run the same `dnsmasq` and `nginx` configurations the Controller profile uses, generated by the same code, from a clearly labelled bootstrap directory.
- On a network that already has a DHCP authority, run dnsmasq in **ProxyDHCP** mode so it supplies only boot options and never addresses. Two address- assigning authorities on one layer-2 network remain forbidden by ADR 0008.
- The bootstrap host builds the Archiso image and publishes the artifact manifest.
- Bootstrap state is temporary. After the Controller passes acceptance, the bootstrap services are stopped and the Controller serves subsequent installs.

The bootstrap host must not later be provisioned as a managed machine while it is the only source of the boot chain.

ADR 0053 Own day-two configuration with Ansible from this repository
Accepted · 2026-07-25

Split the lifecycle in two. **Install time** does only what cannot be done later: partition, encrypt, create the filesystem, install the base system and kernels, write the boot artifacts, configure the managed interface and static address, install `sshd` with the administrator public key, and record the manifest. Nothing else. **Converge time** is Ansible, run over SSH from this repository, and owns everything else: package sets, dnsmasq and nginx configuration, the services role, identity client configuration, users, sudo policy, health checks and update orchestration.

- Inventory lives in the private overlay; roles and playbooks are public.
- Every role must be idempotent and must pass `--check` cleanly against a converged host.
- No secret is stored in a playbook. Secret material is referenced, and the mechanism is chosen in a later ADR.
- The Controller converges itself the same way a Workstation does. There is no privileged manual path.

ADR 0054 Define an optional Services role for always-on applications
Accepted · 2026-07-25

Define a third role, `services`, separate from Controller and Workstation.

- Implement each application as a Podman **quadlet** — a systemd unit generated from a declarative container definition — so lifecycle, ordering, restart policy and journal integration are ordinary systemd.
- Applications that need discovery run with host networking, which is required for HomeKit and UPnP to work at all.
- USB radios are bound by stable `by-id` path and declared per application.
- Application state lives on its own Btrfs subvolume, excluded from the operating-system checkpoint under ADR 0027 and backed up separately, because its restore point is not the same as the operating system's.
- The role is **disabled by default** and enabled per instance.
- The role may be assigned to the Controller host today. Moving it to its own machine later changes which host claims the role and nothing else.

The Controller profile does not depend on the services role, and the services role does not depend on Controller-hosted DHCP or DNS beyond ordinary name resolution.

ADR 0055 Use Samba AD DC for shared Windows and Linux identity
Accepted · 2026-07-25

Use Samba Active Directory Domain Services as the directory.

- Run it in a dedicated VM, not on the Controller host. It is a distinct trust boundary, it needs its own restart and upgrade cadence, and ADR 0014 permits a VM exactly when those conditions hold.
- Arch clients use SSSD's AD provider with credential caching.
- Windows Pro clients domain-join. Windows Home cannot and keeps local accounts.
- Home directories stay local; `pam_mkhomedir` creates them on first login.
- Every managed machine keeps a separately named local break-glass administrator with its own sudo rule, and that account is never a directory account.
- UID and GID come from the directory. The existing local `ksh` at UID 1000 is migrated deliberately and only after piloting with a separate test identity.
- Set an explicit offline credential lifetime longer than a normal trip rather than relying on a default.

A second domain controller on separate physical hardware is required before the directory is considered production, and is deferred until the first one works.

ADR 0056 Prove every change in a QEMU/OVMF matrix before hardware
Accepted · 2026-07-25

Maintain a QEMU/OVMF acceptance matrix in `homelab/qemu/`, runnable with one command. The matrix builds a virtual isolated network with no router and boots:

- a virtual **bootstrap** host serving ProxyDHCP, TFTP and HTTP;
- a virtual **Controller** target, installed from the real installer with no human present; ADR 0058 defines the mechanism, which drives the genuine interactive prompts through a pseudo-terminal rather than adding an unattended code path; and
- a virtual **Workstation** target that pivots on the Controller's FQDN.

It then asserts, automatically:

- the installer refuses every invalid network plan in its rejection corpus;
- the Controller installs, powers off, and activates services on first boot;
- a client receives a lease inside the configured pool and never the Controller address;
- `home.arpa` resolves the Controller;
- **no default route is advertised**, per ADR 0011;
- the artifact manifest checksums verify;
- Ansible converge is idempotent — a second run reports no changes; and
- a checkpoint can be created, rolled back to, and validated.

Physical installation is permitted only after the matrix passes. UEFI variables use a per-run OVMF variable store so firmware state never leaks between runs.

ADR 0057 Record corrections to the deferred signing design
Accepted · 2026-07-25

Do not act on this ADR during Milestone A. When ADRs 0029 through 0042 are revalidated, treat the following as required inputs. **1. The key separation in ADRs 0033 to 0035 is largely notional.** ADR 0033 makes the TPM policy key cryptographically distinct from the Secure Boot key, which is correct. ADR 0035 then stores both on the same removable medium and ADR 0039 gives the primary and backup media the same passphrase. One compromised passphrase yields both identities on both media, so the separation survives on paper only. Either separate the custody or stop paying for the separation. **2. ADR 0039's shared passphrase defeats the backup's independence.** A backup exists to survive a failure the primary did not. Sharing its passphrase means a credential compromise takes both copies simultaneously. **3. ADR 0030's additive trust model may be infeasible on the actual hardware.** Many consumer and OEM firmwares permit `db` modification only after entering Setup Mode, which clears PK and therefore replaces platform ownership — the exact thing ADR 0030 forbids. ADR 0030 handles this by failing preflight, which means the profile may simply not support the machine. Before Milestone B, test enrollment on the real target firmware and decide what happens when it cannot be done additively. Also revisit whether three signed UKIs and two signing identities are proportionate to a single-operator homelab, given the operational cost the custody chain in ADRs 0036 through 0042 imposes.

ADR 0058 Drive acceptance tests through the real interactive installer
Accepted · 2026-07-26

No unattended installation code path exists. The installer is interactive and only interactive. The acceptance matrix drives it by attaching to a pseudo-terminal and answering prompts the way a person would, including the final confirmation.

- Prompt definitions live in one registry, `homelab/lib/prompts.py`: each prompt has a stable identifier, its human-facing text, its validator and its help text.
- The installer renders that registry. The test harness imports the same registry to know what to expect. There is one source of truth, so changing a prompt's wording cannot silently desynchronize the tests.
- The harness matches on prompt identity, supplies an answer, and asserts on what the installer prints back.
- There is no flag, environment variable or file that skips a prompt, and no code path that reaches a destructive operation without the confirmation having been answered.

The final confirmation requires the operator to **type the target disk's serial number**, not to answer yes. A serial cannot be typed from muscle memory, cannot be answered correctly by someone who has not read the summary, and cannot be satisfied by a stray keystroke.

ADR 0059 Fail fast, leave the evidence, do not roll back or resume
Accepted · 2026-07-26

Stop at the failing step. Change nothing further. Report precisely.

- No rollback. A cleanup path is a second destructive code path that only ever executes when something is already wrong, and running it destroys the state that explains the failure.
- No resume. Resume logic must reason correctly about state it did not observe, which is where installers grow their worst bugs. A re-run starts from the beginning.
- On failure the installer prints: the steps that completed, the step that failed, the exact command, its exit status, and its captured output; then states plainly that the disk is **not bootable** and that the fix is to run the installer again.
- The installer exits non-zero so the acceptance harness can detect it.

Steps are declared as either non-destructive or destructive. **A destructive step cannot execute without an authorization token**, and that token can only be produced by the confirmation described in ADR 0058. This is structural, not a convention: there is no boolean to pass by mistake.

ADR 0060 Record a non-secret manifest on the installed system
Accepted · 2026-07-26

Write one JSON manifest to `/etc/homelab/manifest.json` on the installed root, and echo it to the console at the end of the run.

- The machine carries its own provenance. A Controller found in a cupboard in three years can state what it is, when it was built, and from what.
- The console copy is how the acceptance harness under ADR 0056 captures the manifest without mounting an encrypted filesystem.
- Nothing is uploaded during installation. There is no service to upload to when the first Controller is built, and adding one would make provisioning depend on it.
- The manifest is **non-secret by construction** and is validated against that before being written. It records identities, not credentials: no passphrase, no recovery key, no private key, no token, no LUKS header, no password hash.
- It is not written to the ESP. ADR 0020 keeps the ESP minimal and secret-free, and interface MACs, disk serials and network plans are exactly the instance data ADR 0046 keeps out of anything published or casually readable.

Recorded: schema version; installer version and the artifact checksums it verified; timestamp; profile; hostname; whether this was a `development-proof` installation; firmware mode, Secure Boot state and TPM presence as *observed*; target disk path, model, serial and size; managed interface name and permanent MAC; the confirmed network inputs and every derived value; and the partition layout actually written.

ADR 0061 Test the disk-writing steps against loopback devices
Superseded by ADR 0062 · 2026-07-26

Write the disk steps so they take a **device path** and nothing else about how that device came to exist. A test can then supply a loopback device backed by a sparse file, and the genuine `sgdisk`, `cryptsetup` and `mkfs.btrfs` run against it for real.

- Loopback tests live in `homelab/tests/integration/` and are **not** part of the default suite. They need root and they are slow.
- Run them with an explicit target: `make homelab-integration`.
- Each test creates its own sparse backing file in a temporary directory, attaches it, runs the step, asserts on the result by re-reading the device with `sgdisk --print`, `cryptsetup luksDump` and `btrfs subvolume list`, and detaches and deletes it unconditionally.
- A test must never touch a device it did not create. The helper refuses any path that is not a loop device it attached itself, and the refusal is tested.
- `pacstrap` and UKI generation are **not** covered here. They need network access and a full Arch environment, so they belong to the QEMU matrix under ADR 0056.

This does not replace the acceptance matrix. Loopback tests answer "does this step produce the layout it claims"; the matrix answers "does a machine built this way boot and serve DHCP".

ADR 0062 Reach a running install cycle before hardening the installer
Accepted · 2026-07-26

Reach a running install cycle first. Specifically:

- ADR 0061 is superseded. There is no loopback test tier.
- `disks.py` provides the commands to run and nothing else. The `verify_*` functions that re-read and parse `sgdisk`, `cryptsetup` and `btrfs` output are removed; in the QEMU matrix a wrong layout presents as a machine that does not boot, within about ninety seconds.
- Priority order is now: installer entry point, Archiso profile, QEMU matrix, then a real install on the spare machine.
- Hardening resumes after the cycle exists, directed by what actually fails.

Retained deliberately, because retrofitting them is expensive and they are what stands between an operator error and an erased disk:

- the validation layer —- `netplan`, `prompts`, `preflight`;
- the structural authorization token in `steps.py`; and
- the manifest's non-secret guarantee.

ADR 0063 Use a dedicated key pair for the break-glass administrator
Accepted · 2026-07-26

The break-glass administrator is authorized by a **dedicated key pair used for nothing else**.

- The operator generates it. Nothing in this repository handles the private half, generates it, copies it, or knows where it lives.
- The public key is recorded in the gitignored instance overlay (`group_vars/all.yml`), never in Git and never in a published document.
- The private key is stored where the operator keeps private keys and backed up somewhere that does not depend on the homelab being reachable. A break-glass key stored only on a homelab machine is not a break-glass key.
- The `common` role refuses to converge a machine when no break-glass key is configured, and `identity_client` refuses to join a machine to the directory for the same reason. Both are assertions, not warnings.
- The key is not used for ordinary administration. Ordinary administration is Ansible convergence under a directory identity once one exists.